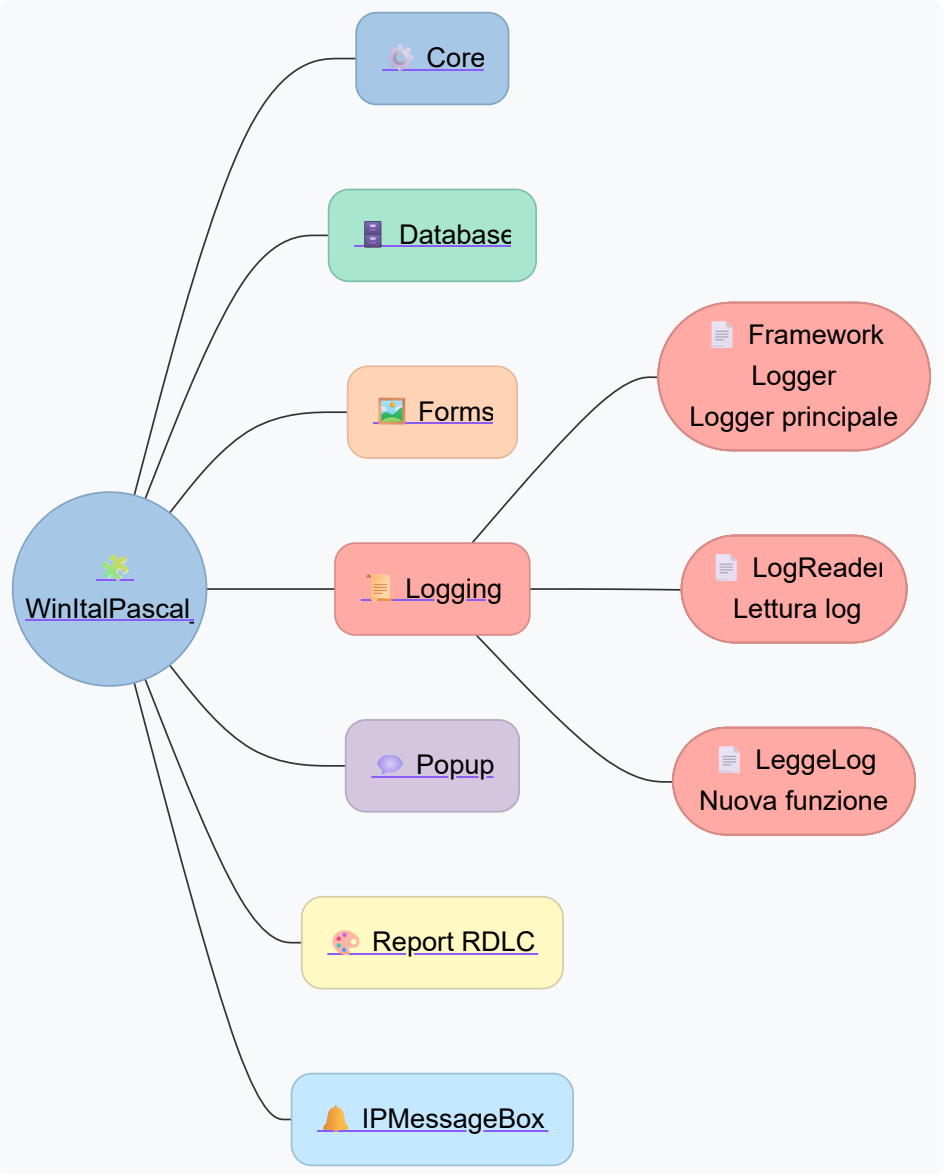
Ital Pascal


Core
Database
Forms
Framework
Logger
Logger principale
WinItalPascal
Logging
LogReader
Lettura log
Popup
LeggeLog
Nuova funzione
Report RDLC
IPMessageBox

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi del codice `FrameworkLogger`

1. Scopo generale

Il file definisce una classe **singleton** (`NotInheritable`) chiamata `FrameworkLogger` che fornisce un semplice meccanismo di logging per scrivere messaggi di testo in un file di log situato nella cartella di esecuzione dell'applicazione.
Il logger è pensato per essere utilizzato in modo statico, senza creare istanze, e garantisce che eventuali errori di scrittura non interrompano l'esecuzione dell'applicazione.

---

2. Componenti principali

Elemento	Descrizione	Note
`Private Sub New()`	Costruttore privato.	Impedisce la creazione di istanze; la classe è "singleton" non-instanziabile.
`Private Shared ReadOnly LogFile As String`	Percorso completo del file di log.	Costruito con `Path.Combine` per garantire la corretta separazione dei componenti del percorso.
`Public Shared Sub Log(message As String)`	Metodo pubblico per scrivere un messaggio.	Utilizza `File.AppendAllText` per aggiungere una nuova riga al file.
`Public Shared Sub LogError(ex As Exception, Optional context As String = "")`	Metodo per registrare un errore.	Chiama `Log` con un formato predefinito che include il contesto e il messaggio dell'eccezione.

Dettagli di implementazione

- **Formato della riga di log**

```
$"{DateTime.Now:yyyy-MM-dd HH:mm:ss} - {message}
{Environment.NewLine}"
```

  - `DateTime.Now` viene formattato in `yyyy-MM-dd HH:mm:ss`.
  - Viene aggiunto il messaggio passato e una nuova riga (`Environment.NewLine`).
- **Gestione degli errori**
  Il blocco `Try…Catch` all'interno di `Log` è vuoto: in caso di eccezione (ad es. file in uso, permessi insufficienti) il logger non fa nulla, evitando che il fallimento del logging propaghi un crash all'applicazione principale.
- **Metodo `LogError`**
  Accetta un oggetto `Exception` e un contesto opzionale. Il messaggio loggato è di forma:

```
ERROR [context] - <messaggio eccezione>
```

---

3. Flusso di esecuzione

1. **Inizializzazione**
   - Al primo accesso a `FrameworkLogger`, il compilatore crea la variabile `LogFile` con il percorso del file di log.
2. **Scrittura di un messaggio** (`Log`)
   - Viene chiamato `File.AppendAllText` con la stringa formattata.
   - Se la scrittura va a buon fine, la riga viene aggiunta al file.
   - Se si verifica un errore, il blocco `Catch` intercetta l'eccezione e non fa nulla (silenzio).
3. **Registrazione di un errore** (`LogError`)
   - Costruisce un messaggio di errore con il contesto e il messaggio dell'eccezione.
   - Chiama `Log` per scrivere la riga di log.

> ### 🔹 Conclusione
>
> Il `FrameworkLogger` è un semplice, ma efficace, strumento di logging che consente di scrivere messaggi e errori in un file di testo senza rischiare di interrompere l'applicazione. È ideale per progetti di piccole o medie dimensioni dove un logging avanzato non è necessario.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi del codice `LogLeggiScrivi`

1. Scopo generale

Il file definisce una classe `LogLeggiScrivi` che fornisce un semplice sistema di logging per un'applicazione Windows Forms.

- **Scrittura**: aggiunge eventi, eccezioni o messaggi generici a un file di log (`ItalPascal_Log.txt`) situato nella cartella di esecuzione dell'applicazione.
- **Lettura**: apre il file di log con l'applicazione predefinita del sistema operativo.
- **Pulizia**: mantiene solo gli ultimi *N* eventi (default 5) eliminando il resto.

La classe è `NotInheritable` (finale) e tutti i metodi sono `Shared`, quindi non è necessario creare un'istanza per usarla.

---

2. Componenti chiave

Metodo	Funzione	Parametri	Note
`ApriLog(Optional NomeFileLog As String = NomeFileLogDefault)`	Apre il file di log con l'applicazione di default.	`NomeFileLog` (opzionale)	Rest il file `Fal`
`ScriviLog(Procedura As String, ex As Exception)`	Scrive un'eccezione dettagliata.	`Procedura` (nome della routine), `ex` (eccezione)	Aggi sepa ever
`ClearLog(Optional NumeroEventiDaMantenere As Integer = 5)`	Mantiene solo gli ultimi *N* eventi.	`NumeroEventiDaMantenere`	Gli e delir inizi `--`

Metodo	Funzione	Parametri	Note
`ScriviLogMsg(Messaggio As String)`	Scrive un messaggio generico.	`Messaggio`	Utiliz non
Costanti	`NomeFileLogDefault`	Stringa con il nome del file di log.	Il va `Ita`

Struttura del file di log

Ogni evento è delimitato da una riga di 80 caratteri (`=` o `-`).
Un evento di eccezione contiene:

```
Data       : dd/MM/yyyy HH:mm:ss
Procedura  : <nome>
Messaggio  : <ex.Message>

Dettagli eccezione:
<ex.ToString()>
```

Un evento di messaggio contiene:

```
Data       : dd/MM/yyyy HH:mm:ss
Evento     : <Messaggio>
```

---

3. Flusso di esecuzione

3.1 Apertura del log (`ApriLog`)

1. **Costruzione del percorso**: `Path.Combine(AppDomain.CurrentDomain.BaseDirectory, NomeFileLog)`.
2. **Verifica esistenza**: se il file non esiste, mostra un `MessageBox` di avviso e restituisce `False`.
3. **Apertura**: `Process.Start` con `UseShellExecute = True` apre il file con l'applicazione predefinita (es. Notepad).
4. **Gestione eccezioni**: eventuali errori mostrano un `MessageBox` di errore e restituiscono `False`.

3.2 Scrittura di un'eccezione (`ScriviLog`)

1. **Percorso**: come sopra, ma sempre con `NomeFileLogDefault`.
2. **StreamWriter**: aperto in modalità append (`True`).
3. **Scrittura**: header, data, procedura, messaggio, dettagli dell'eccezione.
4. **Try/Catch**: in caso di errore di scrittura, l'eccezione viene ignorata (commento "Evita errori durante il logging").

3.3 Scrittura di un messaggio (`ScriviLogMsg`)

1. **Percorso**: come sopra.
2. **StreamWriter**: modalità append.
3. **Scrittura**: header, data, evento.
4. **Try/Catch**: idem a `ScriviLog`.

3.4 Pulizia del log (`ClearLog`)

1. **Percorso**: come sopra.
2. **Legge tutte le righe** con `File.ReadAllLines`.
3. **Separazione degli eventi**:
   - Quando una riga inizia con `===` o `---`, si considera l'inizio di un nuovo evento.
   - Le righe vengono raccolte in una lista di liste (`eventi`).
4. **Selezione degli ultimi N eventi**: `eventi.Skip(eventi.Count - NumeroEventiDaMantenere)`.
5. **Scrittura**: apre un nuovo `StreamWriter` in modalità overwrite (`False`) e scrive solo gli eventi selezionati.
6. **Gestione eccezioni**: mostra un `MessageBox` in caso di errore.

---

4. Possibili utilizzi

Il frammento mostra come **scrivere messaggi di log** in un'applicazione VB.NET. Il logger è fornito da una classe chiamata `LogLeggiScrivi` con il metodo `ScriviLogMsg(String)`. I messaggi registrati possono essere usati per tracciare l'avvio dell'applicazione, operazioni di salvataggio, cancellazioni o per diagnosticare eventuali errori.

```
    ' Esempi di utilizzo:
     LogLeggiScrivi.ScriviLogMsg("Applicazione avviata.")
     LogLeggiScrivi.ScriviLogMsg("Anagrafica cliente salvata correttamente.")
    LogLeggiScrivi.ScriviLogMsg($"Salvati {dgvClienti.Rows.Count} record.")
    LogLeggiScrivi.ScriviLogMsg($"Cliente ID {IDCliente} eliminato.")


     Dim Utente As String = FatDataGrid.CurrentRow.Cells("5").Value.ToString()
     Dim NumeroFattura As String = FatDataGrid.CurrentRow.Cells("3").Value.ToString()
     LogLeggiScrivi.ScriviLogMsg( $"Fattura {NumeroFattura}"& "del Sig. $"{Utente}" & "salvata correttamente.")
```

---

1. **Costruzione e log del messaggio di fattura**

```
    LogLeggiScrivi.ScriviLogMsg( $"Fattura {NumeroFattura}"& "del Sig. "
& $"{Utente}" & "salvata correttamente.")
```

- Si tenta di concatenare una stringa interpolata con altre parti usando l'operatore `&`.
- Il risultato finale dovrebbe essere:

`Fattura 1234 del Sig. Mario Rossi salvata correttamente.`

---

5. Come usarlo

```vb
' Apri il log (mostra il file)
LogLeggiScrivi.ApriLog()

' Scrivi un messaggio semplice
LogLeggiScrivi.ScriviLogMsg("Applicazione avviata.")

' Scrivi un'eccezione
Try
    ' codice che può generare eccezione
Catch ex As Exception
    LogLeggiScrivi.ScriviLog(NameOf(CaricaDati), ex)
End Try

' Pulisci il log mantenendo solo gli ultimi 10 eventi
LogLeggiScrivi.ClearLog(10)
```

> ✏️ 🔹 **Conclusione**
>
> La classe `LogLeggiScrivi` è un helper semplice ma completo per la gestione di file di log in un'applicazione VB.NET. Fornisce metodi statici per aprire, scrivere e pulire il log, con una gestione di errore robusta e una struttura di file leggibile.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi del codice `LogReader`

1. Scopo generale

Il file `LogReader.vb` è una classe **utility** (non ereditabile) che fornisce metodi statici per gestire un file di log di nome `ItalPascal_Log.txt`.
Questa classe permette di:

- Leggere l'intero contenuto del log.
- Aprirlo con Notepad.
- Cancellarlo (eliminare il file).

Tutto il lavoro è svolto in modo **thread-safe** e con gestione degli errori tramite `Try/Catch`.

---

2. Componenti chiave

Sezione	Descrizione	Funzione principale
`Imports`	Importa i namespace necessari (`System.IO`, `System.Diagnostics`, `System.Windows.Forms`).	Fornisce accesso a file, processi e UI.
`Public NotInheritable Class LogReader`	Classe finale, non estendibile.	Contiene tutti i metodi di utilità.
`Private Sub New()`	Costruttore privato.	Impedisce l'istanza della classe (tutti i metodi sono statici).
`Private Shared ReadOnly LogFile As String`	Costante che costruisce il percorso completo del file di log.	Centralizza la posizione del file.

Sezione	Descrizione	Funzione principale
`Public Shared Function ReadLog() As String`	Legge l'intero file di log.	Restituisce il contenuto o un messaggio di errore.
`Public Shared Sub OpenLog()`	Avvia Notepad con il file di log.	Permette all'utente di visualizzare il log in un editor di testo.
`Public Shared Sub ClearLog()`	Elimina il file di log se esiste.	Pulisce lo spazio di archiviazione.
Vedi LogLeggiScrivi	Suggerimenti su come usare la classe.	Fornisce esempi di utilizzo.

Elemento	Funzione
`Path.Combine(...)`	Unisce in modo sicuro più segmenti di percorso, aggiungendo i separatori di directory corretti per il sistema operativo
`Application.StartupPath`	Restituisce la cartella in cui è stato avviato l'eseguibile (es. `C:\Programmi\ItalPascal\`)
`"ItalPascal_Log.txt"`	Specifica il nome del file che verrà creato o scritto

---

3. Flusso di esecuzione

3.1 Costruzione del percorso del file

```
Private Shared ReadOnly LogFile As String =
    Path.Combine(
        Application.StartupPath,
        "ItalPascal_Log.txt")
```

- `Application.StartupPath` restituisce la cartella in cui è stato avviato l'applicazione.
- `Path.Combine` crea un percorso completo e sicuro, indipendentemente dal sistema operativo.

3.2 Lettura del log (`ReadLog`)

1. **Verifica esistenza**: `If File.Exists(LogFile)`.
2. **Lettura**: `File.ReadAllText(LogFile)` restituisce il contenuto come stringa.
3. **Fallback**: Se il file non esiste, ritorna `"Nessun log disponibile."`.
4. **Gestione eccezioni**: In caso di errore, restituisce `ex.Message`.

3.3 Apertura del log (`OpenLog`)

1. **Verifica esistenza**: `If File.Exists(LogFile)`.
2. **Avvio Notepad**: `Process.Start("notepad.exe", LogFile)`.
3. **Messaggio di errore**: Se il file non è presente, mostra un `MessageBox`.
4. **Gestione eccezioni**: Mostra eventuali errori con `MessageBox.Show(ex.Message)`.

3.4 Cancellazione del log (`ClearLog`)

1. **Verifica esistenza**: `If File.Exists(LogFile)`.
2. **Eliminazione**: `File.Delete(LogFile)`.
3. **Gestione eccezioni**: Mostra eventuali errori con `MessageBox.Show(ex.Message)`.

---

4. Come usarla

```
' Leggi e mostra il log
MessageBox.Show(LogReader.ReadLog(), "FILE LOG")

' Apri il log con Notepad
LogReader.OpenLog()

' Cancella il log
LogReader.ClearLog()
```

> 🔹 **Conclusioni**
>
> Questa classe è ideale per piccoli progetti in cui si desidera un semplice sistema di logging senza dipendenze esterne.
> Per applicazioni più complesse, si consiglia di utilizzare librerie di logging (es. NLog, log4net) che offrono funzionalità avanzate.